

MACHINE LEARNING FUNDAMENTALS SERIES

Long Short-Term Memory (LSTM): Principles & Architecture

Dual-Path Memory, Sigmoid vs. Tanh Activations, Gate Mechanics, and Sequence Learning

1. Motivation and the Origin of "Long Short-Term Memory"

In standard Vanilla Recurrent Neural Networks (RNNs), processing sequential data over extended time steps (e.g., 50 days of stock market data) leads to severe **Vanishing or Exploding Gradients**. Because feedback signals pass through repeated matrix weight multiplications (W_2^N), gradients either explode ($2^{50} \approx 10^{15}$) or vanish ($0.5^{50} \approx 10^{-15}$), rendering the network incapable of retaining long-term context.

1.1 Why is it Called "Long Short-Term Memory"?

Standard RNNs possess only a short-term hidden state that decays rapidly over time steps. The architecture is named **Long Short-Term Memory (LSTM)** because it embeds a **Long-term memory** path (the Cell State) directly into the **Short-term memory** recurrent framework of an RNN, allowing short-term memory units to persist information over long time horizons.

1.2 The Dual-Path Memory Paradigm

Instead of relying on a single feedback loop for both ancient history and immediate past events, an LSTM unit utilizes **two separate memory paths**:

- **Cell State (C_t - Long-Term Memory)**: Represented by the line running across the top of the unit. Crucially, the Cell State is **not directly multiplied by weights and biases along its main line**. It is modified only by linear additions and element-wise scaling, allowing long-term memories to flow across unrolled time steps without vanishing or exploding gradients.
- **Hidden State (h_t - Short-Term Memory)**: Represented by the line running across the bottom of the unit. It is directly connected to learnable weights and biases and serves as the **final output of the entire LSTM unit** at each time step.

2. Activation Functions: Sigmoid vs. Tanh

LSTMs rely on two complementary activation functions to control memory operations:

Activation	Mathematical Formula	Output Range	Functional Role in LSTM
Sigmoid (σ)	$\sigma(x) = \frac{1}{1+e^{-x}}$	(0.0, 1.0)	Gate Keeper / Percentage : Determines what percentage (0% to 100%) of memory to keep or forget.
Tanh	$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$	(-1.0, 1.0)	Candidate Memory / Rescaling : Creates potential memories or rescales values centered around 0.

- **Sigmoid as a Percentage Gate**: Multiplying a memory vector by 0.997 retains 99.7% of the memory; multiplying by 0.0 completely erases it ($x \times 0 = 0$).
- **Tanh as a Value Generator**: Because tanh maps inputs to $[-1, 1]$, it allows the network to increase (positive values) or decrease (negative values) memory states in a balanced, stable range.

3. LSTM Terminology and Gate Mechanics

An LSTM unit processes inputs (x_t), previous short-term memory (h_{t-1}), and previous long-term memory (C_{t-1}) through three sequential stages (Gates):

3.1 Stage 1: Forget Gate (Filtering Long-Term Memory)

The **Forget Gate** determines what percentage of the previous long-term memory (C_{t-1}) should be remembered:

1. Combines short-term memory (h_{t-1}) and current input (x_t) with weights and biases:

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \in (0, 1)$$

]Multiplies C_{t-1} by f_t : $C_{\text{retained}} = C_{t-1} \odot f_t$.

3.2 Stage 2: Input Gate (Adding New Long-Term Memory)

The **Input Gate** constructs a potential new long-term memory and decides how much of it to add:

2. **Potential Long-Term Memory:** Uses tanh to combine h_{t-1} and x_t into candidate memory $\tilde{C}_t$:

$$\tilde{C}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_c) \in (-1, 1)$$

]Percentage to Add : Uses Sigmoid to compute retention factor i_t :

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \in (0, 1)$$

]

2. **Updated Long-Term Memory:**

$$C_t = (C_{t-1} \odot f_t) + (i_t \odot \tilde{C}_t)$$

3.3 Stage 3: Output Gate (Updating Short-Term Memory & Unit Output)

The **Output Gate** produces the new short-term memory (h_t), which also acts as the **final output** of the LSTM unit:

1. **Potential Short-Term Memory:** Passes the updated long-term memory C_t through tanh:
Potential $h_t = \tanh(C_t) \in (-1, 1)$

2. **Percentage to Pass:** Computes output gate scale o_t using Sigmoid:

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \in (0, 1)$$

]New Short-Term Memory / Final Output :

3. $h_t = o_t \odot \tanh(C_t)$

Crucial Answer: Short-Term vs. Long-Term Output

The Short-Term Memory (h_t / Hidden State) is the FINAL OUTPUT of the LSTM unit. Although the Long-Term Memory (C_t / Cell State) carries context across long time spans, it remains internal to the unit. The Short-Term Memory h_t is passed to downstream prediction layers and to subsequent time steps.

4. Sequence Prediction Mechanics: Company A vs. Company B Example

To understand how LSTMs learn and predict patterns, consider stock prices for two companies over 5 days:

- **Company A:** Day 1 = 0.0, Day 2 = 0.5, Day 3 = 0.5, Day 4 = 0.5, Day 5 = 0.0.
- **Company B:** Day 1 = 1.0, Day 2 = 0.5, Day 3 = 0.5, Day 4 = 0.5, Day 5 = 1.0.

4.1 How the Model Learns the Data Dependency

Days 2, 3, and 4 are **identical** for both companies. To correctly predict Day 5, the model **must remember Day 1** across Days 2, 3, and 4:

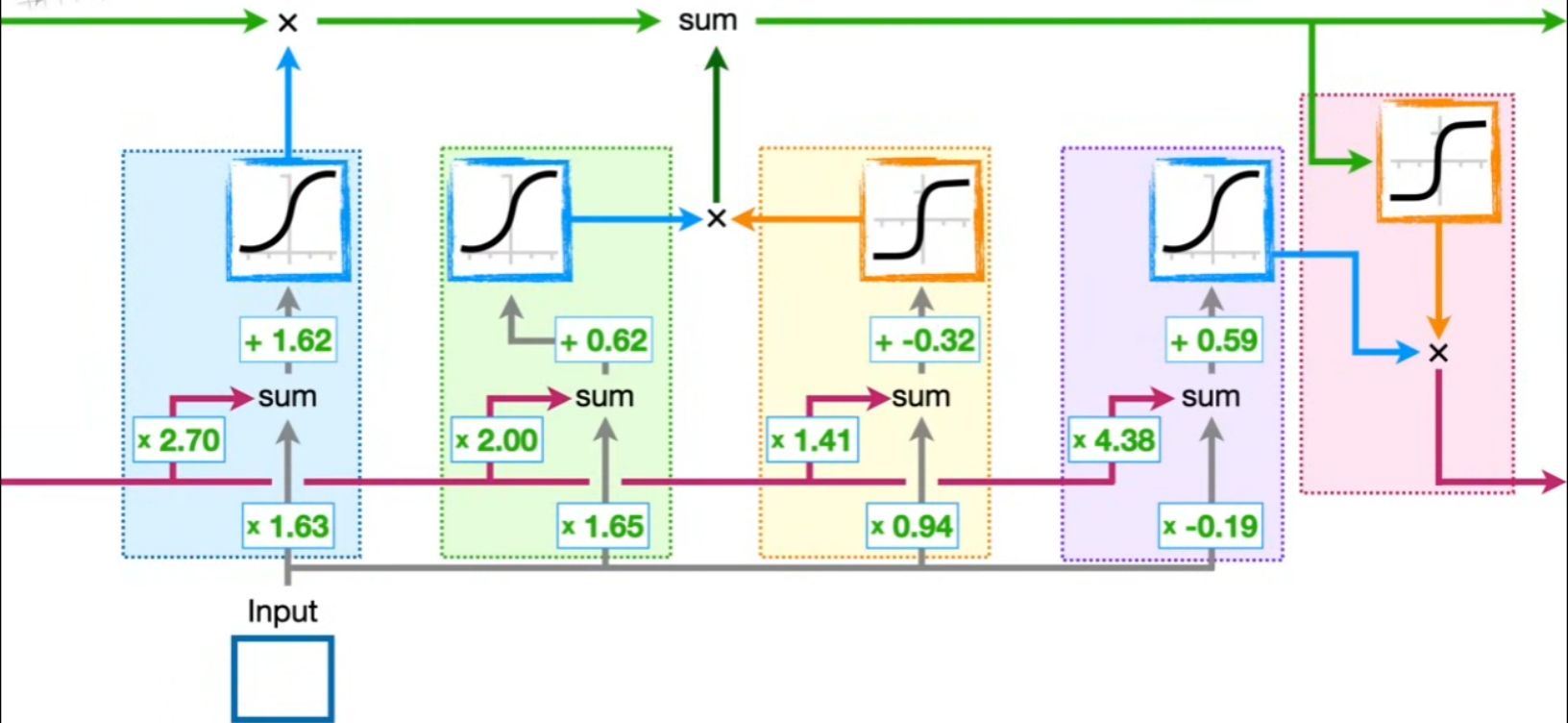
1. **Day 1 Processing:** On Day 1, Company A ($x_1 = 0$) and Company B ($x_1 = 1$) set different long-term memories ($C_1 = -0.20$ for A vs. $C_1 = +0.50$ for B).
2. **Preservation Across Days 2–4:** As Days 2, 3, and 4 ($x = 0.5$) are fed into unrolled units using **shared weights and biases**, the Forget Gate preserves the sign/context of Day 1 in C_t without let weight multiplication decay it to zero.
3. **Day 5 Prediction:** On Day 4, the final short-term memory (h_4) evaluates to 0.0 for Company A (predicting Day 5 = 0.0) and 1.0 for Company B (predicting Day 5 = 1.0).

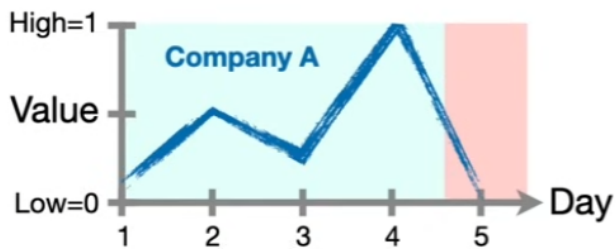
5. Summary of LSTM Gate Architecture

Gate Name	Primary Function	Activation	Key Formula
Forget Gate	Decides % of old C_{t-1} to retain	Sigmoid (σ)	$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$
Input Gate	Creates & decides % of new $\tilde{C}_t$ to add	Tanh & Sigmoid	$\tilde{C}_t = \tanh(\cdot), i_t = \sigma(\cdot)$
Output Gate	Creates new h_t (Final Output)	Tanh & Sigmoid	$h_t = \sigma(\cdot) \odot \tanh(C_t)$



...let's talk about how the **Long Short-Term Memory** unit works.





Initial
Long Term
Memory

Initial
Short
Term
Memory

